Лабораторная работа №3. Основы Docker.

Содержание

Содержание

1 Цели лабораторной работы **3**

2 Задачи к лабораторной работе **3**

3 Методические материалы **4**

3.1 Исторический контекст 4

3.2 Технология Docker 6

3.3 Терминология 6

3.4 Основные команды Docker CLI 7

3.5 Dockerfile 9

3.6 Особенности использования CMD и ENTRYPOINT 11

3.7 Multi-stage сборки 15

4 Ход работы **18**

4.1 Установка Docker 18

4.2 Запуск контейнера nginx 18

4.3 Просмотр логов 18

4.4 Инспектирование контейнера 18

4.5 Описание Dockerfile для python приложения 18

4.6 Описание Dockerfile для java приложения 19

5 Контрольные вопросы **20**

1 Цели лабораторной работы

- Познакомиться с технологией Docker и её концепциями;
- Изучить основные команды Docker CLI;
- Изучить формат описания образов с помощью Dockerfile.

2 Задачи к лабораторной работе

1. Изучить методические материалы к лабораторной работе;
2. Развернуть контейнер nginx:
   - с тегом alpine3.20;
   - с именем *<имя пользователя>-my-nginx*;
   - доступный по порту 8000;
   - запущенный в фоновом режиме;
3. Посмотреть логи приложения;
4. Проинспектировать контейнер и с помощью jq вывести:
   - время, когда контейнер был создан;
   - версию nginx;
   - ip-адрес контейнера;
5. Написать *Dockerfile* для приложения на Python и сбилдить образ;
6. Написать *Dockerfile* для приложения на Java:
   - без использования multi-stage сборки;
   - с использованием multi-stage сборки;
   - сравнить получившиеся образы.

3 Методические материалы

3.1 Исторический контекст

Классическое развёртывание. В начале развития компьютерных технологий код приложений запускался на физических серверах. Не было способа определить границы ресурсов для приложений на хосте, и это вызывало проблемы с распределением ресурсов.

Например, если на физическом сервере работает несколько приложений, может возникнуть ситуация, когда одно приложение будет занимать большую часть ресурсов, в результате чего другие приложения будут работать неэффективно. Решением этой проблемы может быть запуск каждого приложения на отдельном физическом сервере. Но такое решение не подходит для масштабирования, поскольку ресурсы используются не полностью, а содержать много физических серверов слишком дорого.

Развёртывание в виртуальных машинах. В качестве решения была введена виртуализация. Она позволяет запускать несколько виртуальных машин (ВМ) на одном процессоре физического сервера. Виртуализация позволяет изолировать приложения между виртуальными машинами и обеспечивает определенный уровень безопасности, поскольку информация одного приложения не может быть свободно доступна другому приложению.

Виртуализация позволяет лучше использовать ресурсы физического сервера и обеспечивает лучшую масштабируемость, поскольку приложение можно легко добавить или обновить, снижает затраты на оборудование и многое другое. С помощью виртуализации можно представить набор физических ресурсов в виде кластера одноразовых виртуальных машин.

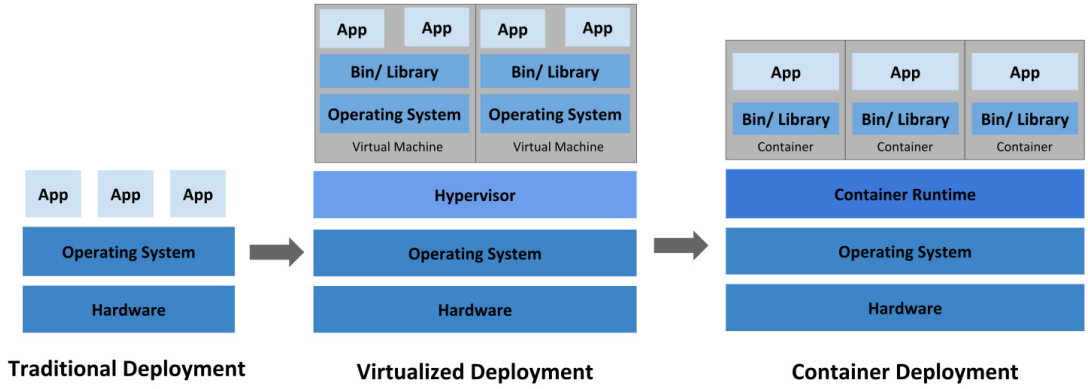
Каждая виртуальная машина - это полноценная машина, работающая со всеми компонентами, включая собственную операционную систему, поверх виртуализированного оборудования.

Развёртывание в контейнерах. Контейнеры похожи на виртуальные машины, но в них ослаблены свойства изоляции, что позволяет разделять операционную систему (ОС) между приложениями. Поэтому контейнеры считаются легковесными.

Подобно ВМ, контейнер имеет собственную файловую систему, долю процессора, памяти, пространства для процессов и т. д. Поскольку они не связаны с базовой инфраструктурой, их можно переносить между облаками и дистрибутивами ОС.

Контейнеры стали популярными, поскольку они обеспечивают дополнительные преимущества, такие как:

- **Ускоренное создание и развертывание приложений** по сравнению с использованием образов VM;
- **Непрерывная разработка, интеграция и развертывание**: обеспечивает надежную и частую сборку и развертывание контейнерных образов с быстрым и эффективным откатом (благодаря неизменяемости образов);
- **Разделение задач Dev и Ops**: создание образов во время сборки/релиза, а не во время развертывания, что позволяет отделить приложения от инфраструктуры;
- **Наблюдаемость**;
- **Переносимость облаков и дистрибутивов ОС**: работает на Ubuntu, RHEL, CoreOS, в локальных сетях, в основных публичных облаках и где угодно еще;
- **Согласованность среды при разработке, тестировании и производстве**
- **Управление, ориентированное на приложения**: повышает уровень абстракции от запуска ОС на виртуальном оборудовании до запуска приложения на ОС с использованием логических ресурсов;
- **Свободно связанные, распределенные, эластичные, освобожденные микросервисы**: приложения разбиваются на более мелкие, независимые части и могут развертываться и управляться динамически;
- **Изоляция ресурсов**: предсказуемая производительность приложений за счёт четкого разграничения ресурсов;
- **Использование ресурсов**: эффективное использование ресурсов.



Эволюция виртуализации

3.2 Технология Docker

Docker - контейнерная платформа программного обеспечения, созданная для разработки, отправки и запуска приложений с использованием контейнерных технологий.

3.3 Терминология

- **Images** (образы) - Схема приложения, которая является основой контейнеров. Образ выступает в роли шаблона, в соответствии с которым создаются контейнеры;

- **Containers** (контейнеры) - Создаются на основе образа и запускают само приложение. Контейнер представляет собой набор процессов, изолированных от других;

- **Docker Daemon** (демон Docker) - Фоновый сервис, запущенный на хост-машине, который отвечает за создание, запуск и уничтожение контейнеров. Демон — это процесс, который запущен на операционной системе, с которой взаимодействует клиент;

- **Docker Client** (клиент Docker) - Утилита командной строки, которая позволяет пользователю взаимодействовать с демоном.

- **Docker Registry** (реестр Docker) - представляет собой удалённую платформу, используемую для хранения и распространения образов.

3.4 Основные команды Docker CLI

- `docker build` — для сборки образа. Инструкции считываются из `Dockerfile`, путь к которому указывается в качестве параметра

  ```
  $ docker build [OPTIONS] <PATH/TO/DOCKERFILE>
  ```

  Основные флаги:

  - `-t` задать имя и тег для образа в формате `name:tag`
  - `-f` указать путь до Dockerfile. По умолчанию ищет файл `./Dockerfile`
  - `--ulimit` установить ограничения ресурсов для контейнера

- `docker run` — запуск контейнера

  ```
  $ docker run [OPTIONS] IMAGE [COMMAND] [ARGS...]
  ```

  Основные флаги:

  - `-d` запустить контейнер в фоновом режиме
  - `-i` запустить контейнер в интерактивном режиме
  - `-t` выделить псевдо-tty
  - `-e` установить переменную окружения `-e ENV=value`
  - `-rm` автоматически удалить контейнер после его остановки
  - `--env-file` задать путь до файла с переменными окружения
  - `--name` задать имя контейнера
  - `-p` прокинуть порт с хоста в контейнер `-p <порт хоста>:<порт контейнера>`

- `docker exec` — выполнить команду внутри запущенного контейнера

  ```
  $ docker exec [OPTIONS] CONTAINER COMMAND [ARGS...]
  ```

  Основные флаги:

  - `-d` запустить контейнер в фоновом режиме
  - `-i` запустить контейнер в интерактивном режиме
  - `-t` выделить псевдо-tty

- `-u` задать имя пользователя, от которого будет выполняться команда (`<name|uid>[:<group|gid>]`)

- `docker images` — вывести список образов

```
$ docker images [OPTIONS]
```

Основные флаги:

- `-a` вывести все образы
- `-q` вывести только ID образов

- `docker ps` — вывести список контейнеров

```
$ docker ps [OPTIONS]
```

Основные флаги:

- `-a` вывести все контейнеры
- `-q` вывести только ID контейнеров

- `docker rm` — удалить один или несколько контейнеров

```
$ docker rm [OPTIONS] CONTAINER [CONTAINER...]
```

- `docker rmi` — удалить один или несколько образов

```
$ docker rmi [OPTIONS] IMAGE [IMAGE...]
```

- `docker inspect` — вывести информацию об объекте Docker

```
$ docker inspect [OPTIONS] NAME|ID [NAME|ID...]
```

- `docker logs` — вывести логи контейнера

```
$ docker logs [OPTIONS] CONTAINER
```

Основные флаги:

- `-f` выводить логи в интерактивном режиме
- `-n` вывести n строк с конца

3.5 Dockerfile

Dockerfile - специальный файл, содержащий набор инструкций, следуя которым Docker будет собирать образ контейнера.

Основные инструкции Dockerfile

Инструкция	Описание	Пример
FROM	Указывает базовый образ	FROM python:3.10
EXPOSE	Порт, прослушивающий приложение	EXPOSE 8080
VOLUME	Точка монтирования	VOLUME /app/storage
WORKDIR	Задать рабочий каталог	WORKDIR /usr/src/app/
ADD	Добавить локальные или удаленные файлы и каталоги	ADD . /usr/src/app/
COPY	Копирование файлов и каталогов	COPY . /usr/src/app/
RUN	Выполнить команды сборки образа	RUN mkdir -p /usr/src/app/
CMD	Указание команд по умолчанию	CMD [’”python”, “main.py”]
ENTRYPOINT	Указание исполняемого файла и аргументов по умолчанию	ENTRYPOINT [”python”, “main.py”]

ПРИМЕЧАНИЕ: обязательной является только инструкция `FROM` (`CMD` и `ENTRYPOINT` используются для образов контейнеров, предназначенных для запуска)

Рассмотрим на примере. Допустим, у нас есть следующая структура проекта:

```
MyApp
| - src
|   | - MyModule
|   |   |   __init__.py
|   |   |   my_module.py
|   |   main.py
|   Dockerfile
|   requirements.txt
```

Структура проекта на хосте

Ниже приведены содержимые файлов (содержанием `__init__.py` и `requirements.txt` можно пренебречь).

```
from MyModule.my_module import write_logs

for i in range(10):
```

```python
    write_logs(i)

print("Done!")
```

main.py

```python
def write_logs(n):
    for i in range(10):
        print(f"#{n}{i}: Log was written successfully!\n")
```

my_module.py

```dockerfile
# В качестве базового образа используем образ python версии 3.10
FROM python:3.10

# Выбираем внутри контейнера рабочую директорию /app
# Следующие команды будут исполнятся из неё
WORKDIR /app

# Копируем файл из папки MyApp на хосте в папку /app контейнера
COPY requirements.txt .

# Устанавливаем зависимости в контейнер
RUN pip install -r requirements.txt

# Переключаемся в директорию /app/src
WORKDIR /app/src

# Копируем папку MyApp/src в папку /app/src контейнера
COPY src .

# При запуске контейнера будет запускаться команда 'python'
# с параметром 'main.py'
ENTRYPOINT ["python", "main.py"]
```

Dockerfile

```
app
| - src
|   | - MyModule
|   |   |   __init__.py
|   |   |   my_module.py
|   |   main.py
|   requirements.txt
```

Структура проекта в контейнере

3.6 Особенности использования CMD и ENTRYPOINT

CMD и ENTRYPOINT нужны для указания контейнеру точки входа, но раз две инструкции, с первого взгляда, выполняют одно и то же, значит в их использовании есть некая особенность.

Особенность №1. Если образ предназначен для запускаемых контейнеров (а не как база для других образов), то как минимум одна из инструкций должна быть определена.

Особенность №2. Если определена только одна из инструкций, то эффект будет одинаковый.

Особенность №3. Для обоих инструкций существует режимы shell и exec.

```
ENTRYPOINT python main.py
# или
CMD python main.py
```

Режим shell

```
ENTRYPOINT ["python", "main.py"]
# или
CMD ["python", "main.py"]
```

Режим exec

Режим shell означает, что 'python main.py' передаётся оболочке (к примеру, /bin/sh) в качестве аргументов при запуске контейнера:

```
/bin/sh -c python main.py
```

Если посмотреть запущенные процессы внутри контейнера, то мы заметим, что процесс '/bin/sh' имеет PID 1, а 'python main.py', отличный от 1:

```
$ docker exec shell_mode ps
PID   USER     TIME   COMMAND
  1   root     0:00   /bin/sh -c python main.py
  7   root     0:00   python main.py
  8   root     0:00   ps
```

Режим exec запускает команду напрямую с PID 1:

```
$ docker exec exec_mode ps
PID   USER     TIME   COMMAND
  1   root     0:00   python main.py
  7   root     0:00   ps
```

Особенность №3.1. Сигналы, отправленные в контейнер, перенаправляются процессу с PID=1. Это означает, что в случае shell сигнал будет обработан оболочкой, а не main.py.

Особенность №3.2. При запуске в режиме exec мы не можем пользоваться переменными среды (к примеру, `$PATH`), а также символами подстановки (к примеру, *.cpp - все файлы с расширением 'cpp').

Особенность №4. Сущствует таблица взаимодействия параметров CMD и ENTRYPOINT.

	No ENTRYPOINT	ENTRYPOINT exec_entry p1_entry	ENTRYPOINT [“exec_entry”, “p1_entry”]
No CMD	*error, not allowed*	/bin/sh -c exec_entry p1_entry	exec_entry p1_entry
CMD [“exec_cmd”, “p1_cmd”]	exec_cmd p1_cmd	/bin/sh -c exec_entry p1_entry	exec_entry p1_entry exec_cmd p1_cmd
CMD [“p1_cmd”, “p2_cmd”]	p1_cmd p2_cmd	/bin/sh -c exec_entry p1_entry	exec_entry p1_entry p1_cmd p2_cmd
CMD exec_cmd p1_cmd	/bin/sh -c exec_cmd p1_cmd	/bin/sh -c exec_entry p1_entry	exec_entry p1_entry /bin/sh -c exec_cmd p1_cmd

Взаимодействие параметров CMD и ENTRYPOINT

Из таблицы можно сделать следующие выводы:

1. При использовании режима shell для ENTRYPOINT аргументы CMD игнорируются.

```
FROM alpine
ENTRYPOINT ls /usr
CMD /proc /home # аргументы будут проигнорированны

# при старте контейнера выполнится: ls /usr
```

2. При использовании режима exec для CMD и ENTRYPOINT аргументы CMD добавляются в конец.

```
FROM alpine
ENTRYPOINT ["ls", "/usr"]
CMD ["/proc", "/home"] # аргументы добавятся в конец

# при старте контейнера выполнится: ls /usr /proc /home
```

3. При использовании режима exec для ENTRYPOINT необходимо использовать режим exec для CMD. Если этого не сделать, Docker попытается добавить sh -c в

уже добавленные аргументы, что может привести к некоторым непредсказуемым результатам.

```
FROM alpine
ENTRYPOINT ["ls", "/usr"]
CMD /proc /home # аргументы будут переданы оболочке и добавлены в конец

# при старте контейнера выполнится: ls /usr /bin/sh -c /proc /home
```

В результате запуска контейнера мы получаем следующий вывод:

```
$ docker run --name=test ent_cmd
/bin/sh

/usr:
bin
lib
local
sbin
share
ls: /proc /home: No such file or directory
```

Если проинспектировать контейнер, то увидим:

```
$ docker inspect test | jq '.[0].Config | .Entrypoint,.Cmd'
[
  "ls",
  "/usr"
]
[
  "/bin/sh",
  "-c",
  "/proc /home"
]
```

Видно, что к аргументам CMD (второй список) неявно добавились /bin/sh и -c, а аргументы, указанные в Dockerfile, слились в один.

Особенность №5. Инструкции ENTRYPOINT и CMD могут быть переопределены с помощью флагов командной строки.

Флаг –entrypoint может быть использован, чтобы переопределить инструкцию ENTRYPOINT:

```
$ docker run --entrypoint [my_entrypoint] some_image
```

Всё, что следует после названия образа в команде docker run, переопределяет инструкцию CMD:

```
$ docker run some_image [arg1] [arg2] [arg3]
```

3.7 Multi-stage сборки

Проблема Docker-образов в том, что каждая инскрукция Dockerfile при сборке создаёт промежуточный слой, который увеличивает финальный размер образа. Эти слои зачастую нужны только в момент сборки, но в финальном образе их быть не должно или их размеры должны быть минимальны.

Начиная с версии 17.05 Docker стал поддерживать многоэтапные сборки (multi-stage builds) для решения этой проблемы. Каждая инструкция FROM может использовать индивидуальный базовый образ и каждая из них начинает новую стадию сборки docker образа. Основное преимущество в том, что появляется возможность копировать необходимые артефакты из одной стадии в другую.

Рассмотрим на примере. Допустим, есть приложение на языке go. Код представляет собой HTTP-сервер, который выводит фразу «Hello, world!»

```
package main;
import (
  "fmt"
  "log"
    "net/http"
)
func main() {
    http.HandleFunc("/helloworld", func(w http.ResponseWriter, r *http.
   Request){
        fmt.Fprintf(w, "Hello, World!")
  })
    fmt.Printf("Server running (port=8080), route: http://localhost
   :8080/helloworld\n")
  if err := http.ListenAndServe(":8080", nil); err != nil {
```

```
      log.Fatal(err)
  }
}
```

Dockerfile

При обычной сборке Dockerfile будет выглядеть следующим образом:

```
# используем образ golang с элиасом "builder"
FROM golang:1.16-buster AS builder
# определяем рабочую папку
WORKDIR /app
# копируем файлы с зависимостями
COPY go.* ./
# устанавливаем зависимости
RUN go mod download
# копируем исходные файлы
COPY *.go ./
# компилируем программу
RUN go build -o /hello_go_http
# ожидаем обращения на 8080 порт
EXPOSE 8080
# указываем в качестве программы для запуска
# ранее скомпилированный файл /hello_go_http
ENTRYPOINT ["/hello_go_http"]
```

Dockerfile

Если собрать такой образ и посмотреть его размер, то увидим 800МБ. Теперь сделаем то же самое, но с multi-stage:

```
# используем образ golang с элиасом "builder"
FROM golang:1.16-buster AS builder
# определяем рабочую папку
WORKDIR /app
# копируем файлы с зависимостями
COPY go.* ./
# устанавливаем зависимости
RUN go mod download
# копируем исходные файлы
COPY *.go ./
# компилируем программу
```

```
RUN go build -o /hello_go_http

# используем образ, содержащий только нужные
# для работы приложения файлы
FROM gcr.io/distroless/base-debian10
# определяем рабочую папку
WORKDIR /
# копируем скомпилированный файл из стадии "builder"
COPY --from=builder /hello_go_http /hello_go_http
# ожидаем обращения на 8080 порт
EXPOSE 8080
# указываем в качестве программы для запуска
# ранее скомпилированный файл /hello_go_http
ENTRYPOINT ["/hello_go_http"]
```

Dockerfile

Собрав и посмотрев размеры образа, мы увидим, что при такой сборке образ весит примерно 25МБ.

4 Ход работы

4.1 Установка Docker

Установите Docker, чётко следуя инструкциям из официальной документации. Для выполнения команд докера без sudo выполните пост-инсталяцию.

4.2 Запуск контейнера nginx

Запустите контейнер nginx в соответствии с требованиями. Вы можете предварительно скачать необходимый образ, используя docker pull <имя_образа:тег>, или сразу указать образ в docker run (по умолчанию docker сначала ищет образы локально и в случае отсутствия начинает поиск в удалённом репозитории).

Убедитесь, что контейнер поднялся, используя команду для вывода контейнеров. Изучите полученный результат. Как только контейнер будет поднят, при обращении по адресу localhost:8000 вы увидите приветственную страницу nginx.

4.3 Просмотр логов

Выведите логи приложения. Посмотрите, как меняется вывод при обновлении приветственной страницы.

4.4 Инспектирование контейнера

jq – мощный инструмент для чтения, фильтрации и записи JSON. Так как информация об объектах докера хранится в JSON-формате, мы будем исользовать jq для поиска необходимых данных.

Установите jq: `sudo apt install jq`

Простой гайд, как пользоваться утилитой можно найти здесь.

ПОДСКАЗКА: Сначала выведите всю информацию о контейнере, изучите вывод и найдите интересующие вас поля. Затем передайте вывод команды `docker inspect` через пайп в jq (используйте флаг -r для вывода "сырым"текстом).

4.5 Описание Dockerfile для python приложения

Склонируйте репозиторий, перейдите в папку python и следуйте инструкциям из README.md.

4.6 Описание Dockerfile для java приложения

Из корневого каталога склонированного репозитория перейдите в папку java и следуйте инструкциям из README.md.

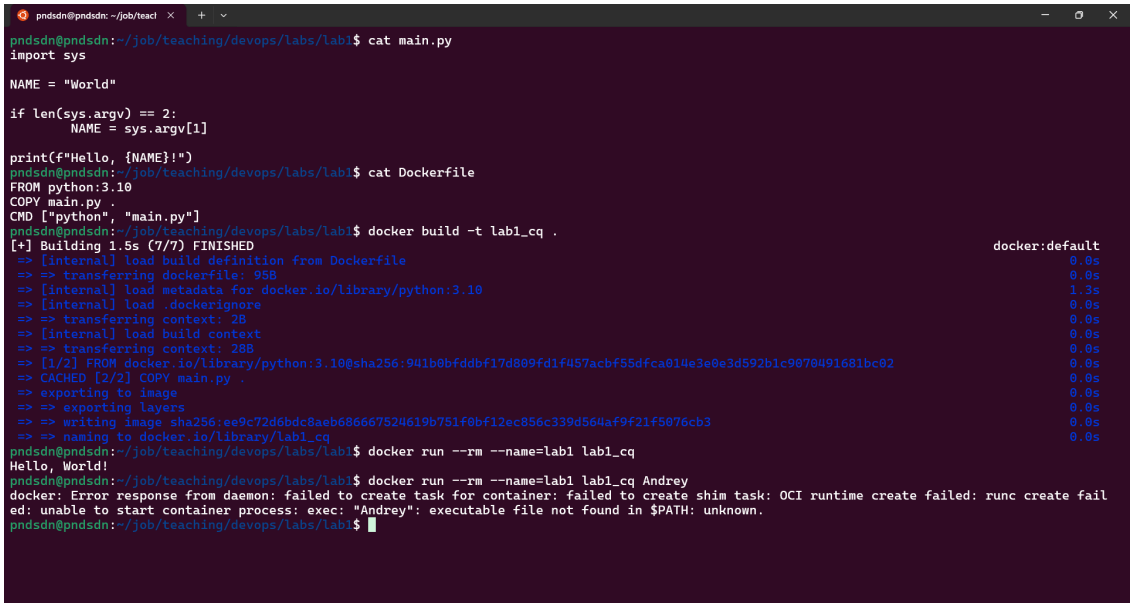
5 Контрольные вопросы

1. Что такое Docker?

2. Что такое образ?

3. что такое контейнер?

4. На рисунке ниже продемонстрирован образ, в котором находится один файл main.py. По умолчанию программа выводит в терминал фразу "Hello, World!". Однако есть возможность изменить приветствие, передав в качестве аргумента слово, которое будет напечатано вместо "World". При втором запуске контейнера произошла ошибка. Укажите на причину ошибки и найдите способ её устранения.



Демонстрация примера